EXP 3 - Measuring Central Tendency and Variability of Data¶

Dataset: Sub_Division_IMD_2017.csv (IMD subdivision-wise monthly rainfall, India)
Use case: KJS-AGR-01 (AI Irrigation Advisory), same dataset as EXP1 and EXP2
Roll No: 16014324020

In [1]:

```
import numpy as np
import pandas as pd
import statistics as st

pd.set_option('display.width', 150)

df = pd.read_csv("Sub_Division_IMD_2017.csv")

# single attribute = ANNUAL rainfall of Konkan & Goa (1901-2017)
goa = df[df['SUBDIVISION'] == 'Konkan &
Goa'].sort_values('YEAR').reset_index(drop=True)
data = goa['ANNUAL'].tolist()
print("Number of years (n):", len(data))
```

```
Number of years (n): 117
```

Task 1: Analysis using built-in functions¶

1a. Single attribute - ANNUAL rainfall (Konkan & Goa)¶

In [2]:

```
# Frequency: every ANNUAL value is different, so I round to the nearest 100 mm first
rounded = pd.Series(data).round(-2).astype(int)
freq = rounded.value_counts().sort_index()
print("Frequency of ANNUAL rainfall (rounded to nearest 100 mm):")
print(freq.to_string())
```

```
Frequency of ANNUAL rainfall (rounded to nearest 100 mm):
1700     1
1900     4
2100     3
2300     1
2400     6
2500     7
2600     3
2700     6
2800    17
2900     7
3000     8
3100    12
3200     8
```

```
3300     7
3400     4
3500     8
3600     5
3700     2
3800     4
3900     2
4000     2
```

In [3]:

```
mean_v = np.mean(data)
median_v = np.median(data)
mode_raw = st.mode(data)
mode_rounded = st.mode(rounded.tolist())
range_v = np.max(data) - np.min(data)
var_v = np.var(data, ddof=1)        # sample variance (n-1)
std_v = np.std(data, ddof=1)
q1, q3 = np.percentile(data, [25, 75])
iqr_v = q3 - q1
cv_v = std_v / mean_v * 100

print("Mean                 :", round(mean_v, 2))
print("Median               :", round(median_v, 2))
print("Mode (raw values)    :", mode_raw, "(every value occurs once)")
print("Mode (rounded 100 mm) :", mode_rounded)
print("Range                :", round(range_v, 2))
print("Variance             :", round(var_v, 2))
print("Standard deviation   :", round(std_v, 2))
print("Q1 / Q3              :", round(q1, 2), "/", round(q3, 2))
print("IQR                  :", round(iqr_v, 2))
print("Coeff. of variation  :", round(cv_v, 2), "%")
```

```
Mean                 : 2987.53
Median               : 2968.5
Mode (raw values)    : 2792.9 (every value occurs once)
Mode (rounded 100 mm) : 2800
Range                : 2317.4
Variance             : 237091.78
Standard deviation   : 486.92
Q1 / Q3              : 2735.5 / 3313.7
IQR                  : 578.2
Coeff. of variation  : 16.3 %
```

Observation: Mean (2987.53) and median (2968.5) are almost the same and the coefficient of variation is only 16.3%, so Konkan & Goa's yearly rainfall is fairly symmetric and stable. Every raw value occurs only once, so the raw mode is not meaningful; after rounding to 100 mm the mode is 2800 mm (17 years).

1b. Multiple attributes - seasonal and annual rainfall (Konkan & Goa)¶

In [4]:

```
cols = ['JF', 'MAM', 'JJAS', 'OND', 'ANNUAL']
multi = goa[cols]
```

```
summary = pd.DataFrame({
    'mean': multi.mean(),
    'median': multi.median(),
    'mode': multi.mode().iloc[0],
    'range': multi.max() - multi.min(),
    'variance': multi.var(),
    'std': multi.std(),
    'IQR': multi.quantile(0.75) - multi.quantile(0.25),
})
summary['CV %'] = summary['std'] / summary['mean'] * 100
print(summary.round(2))
```

```
           mean  median    mode   range   variance     std    IQR    CV %
JF         1.78     0.1     0.0    31.8      18.94    4.35    1.2  244.08
MAM       38.93    15.6     2.9   348.7    3385.37   58.18   38.5  149.45
JJAS    2804.19  2822.7  1460.4  2425.9  210977.31  459.32  543.5   16.38
OND      142.63   121.8   118.8   523.1    9063.96   95.20  110.5   66.75
ANNUAL  2987.53  2968.5  1682.8  2317.4  237091.78  486.92  578.2   16.30
```

Observation: JJAS (monsoon) has almost the same mean, std and IQR as ANNUAL, so the monsoon decides the yearly total. JF has a mean of only 1.78 mm but a CV of 244%, which means winter rainfall is very unpredictable.

Task 2: Same measures WITHOUT in-built functions¶

Only loops and formulas on the ANNUAL rainfall list (no np.mean, st.mode, sorted, sum, max, min).

In [5]:

```
# count the values
n = 0
for x in data:
    n = n + 1

# sort the data by hand (bubble sort), needed for median and quartiles
s = data.copy()
for i in range(n):
    for j in range(n - i - 1):
        if s[j] > s[j + 1]:
            s[j], s[j + 1] = s[j + 1], s[j]

# mean = sum of values / n
total = 0
for x in data:
    total = total + x
mean_m = total / n

# median = middle value of sorted data
mid = n // 2
if n % 2 == 0:
    median_m = (s[mid - 1] + s[mid]) / 2
else:
    median_m = s[mid]
```

```
# mode = value with the highest frequency
counts = {}
for x in data:
    if x in counts:
        counts[x] = counts[x] + 1
    else:
        counts[x] = 1
mode_m = s[0]
best = 0
for value in data:
    if counts[value] > best:
        best = counts[value]
        mode_m = value

# range = highest - lowest (sorted list, so last - first)
range_m = s[n - 1] - s[0]

# variance = sum((x - mean)^2) / (n - 1), std = sqrt(variance)
sq_total = 0
for x in data:
    sq_total = sq_total + (x - mean_m) ** 2
var_m = sq_total / (n - 1)
std_m = var_m ** 0.5

# quartiles using linear interpolation on the sorted list
def percentile(p):
    rank = (p / 100) * (n - 1)
    low = int(rank)
    high = low + 1 if low + 1 < n else low
    frac = rank - low
    return s[low] + frac * (s[high] - s[low])

q1_m = percentile(25)
q3_m = percentile(75)
iqr_m = q3_m - q1_m
cv_m = std_m / mean_m * 100

print(f"{'Measure':<20}{'Manual':>12}{'Built-in':>12}")
print(f"{'Mean':<20}{mean_m:>12.2f}{mean_v:>12.2f}")
print(f"{'Median':<20}{median_m:>12.2f}{median_v:>12.2f}")
print(f"{'Mode (raw)':<20}{mode_m:>12.2f}{mode_raw:>12.2f}")
print(f"{'Range':<20}{range_m:>12.2f}{range_v:>12.2f}")
print(f"{'Variance':<20}{var_m:>12.2f}{var_v:>12.2f}")
print(f"{'Std deviation':<20}{std_m:>12.2f}{std_v:>12.2f}")
print(f"{'IQR':<20}{iqr_m:>12.2f}{iqr_v:>12.2f}")
print(f"{'CV %':<20}{cv_m:>12.2f}{cv_v:>12.2f}")
```

```
Measure                   Manual    Built-in
Mean                     2987.53     2987.53
Median                   2968.50     2968.50
Mode (raw)               2792.90     2792.90
Range                    2317.40     2317.40
Variance               237091.78   237091.78
```

```
Std deviation             486.92      486.92
IQR                       578.20      578.20
CV %                       16.30       16.30
```

Observation: All manual results match the built-in functions up to two decimals, so the formulas are correct. The sort is the costly step in the manual code; mean and variance only need simple loops.

Task 3: Grouped data analysis¶

ANNUAL rainfall of Konkan & Goa grouped into class intervals of 250 mm.

In [6]:

```
width = 250
edges = [1500 + width * i for i in range(12)]      # 1500, 1750, ... 4250

lower, upper, f_list = [], [], []
for i in range(len(edges) - 1):
    lo, hi = edges[i], edges[i + 1]
    f = 0
    for x in data:
        if lo <= x < hi:
            f = f + 1
    lower.append(lo)
    upper.append(hi)
    f_list.append(f)

table = pd.DataFrame({'class': [f"{a}-{b}" for a, b in zip(lower, upper)], 'f':
f_list})
table['midpoint'] = [(a + b) / 2 for a, b in zip(lower, upper)]
table['cum_f'] = table['f'].cumsum()
print(table.to_string(index=False))
```

```
    class  f  midpoint  cum_f
1500-1750  1    1625.0      1
1750-2000  4    1875.0      5
2000-2250  3    2125.0      8
2250-2500  7    2375.0     15
2500-2750 16    2625.0     31
2750-3000 29    2875.0     60
3000-3250 23    3125.0     83
3250-3500 17    3375.0    100
3500-3750  9    3625.0    109
3750-4000  7    3875.0    116
4000-4250  1    4125.0    117
```

In [7]:

```
N = table['f'].sum()
x_mid = table['midpoint'].tolist()
f = table['f'].tolist()
cf = table['cum_f'].tolist()

# mean = sum(f * x) / N
```

```
g_mean = sum(f[i] * x_mid[i] for i in range(len(f))) / N

# median / quartiles = L + ((k*N/4 - CF_before) / f_class) * h
def grouped_quartile(k):
    target = k * N / 4
    for i in range(len(f)):
        if cf[i] >= target:
            cf_before = cf[i - 1] if i > 0 else 0
            return lower[i] + ((target - cf_before) / f[i]) * width

g_median = grouped_quartile(2)
g_q1 = grouped_quartile(1)
g_q3 = grouped_quartile(3)
g_iqr = g_q3 - g_q1

# mode = L + ((f1 - f0) / (2*f1 - f0 - f2)) * h   (modal class = highest frequency)
m = f.index(max(f))
f1 = f[m]
f0 = f[m - 1] if m > 0 else 0
f2 = f[m + 1] if m < len(f) - 1 else 0
g_mode = lower[m] + ((f1 - f0) / (2 * f1 - f0 - f2)) * width

# variance = sum(f * (x - mean)^2) / (N - 1)
g_var = sum(f[i] * (x_mid[i] - g_mean) ** 2 for i in range(len(f))) / (N - 1)
g_std = g_var ** 0.5

print(f"{'Measure':<20}{'Grouped':>12}{'Raw data':>12}")
print(f"{'Mean':<20}{g_mean:>12.2f}{mean_v:>12.2f}")
print(f"{'Median':<20}{g_median:>12.2f}{median_v:>12.2f}")
print(f"{'Mode':<20}{g_mode:>12.2f}{mode_rounded:>12.2f}")
print(f"{'Variance':<20}{g_var:>12.2f}{var_v:>12.2f}")
print(f"{'Std deviation':<20}{g_std:>12.2f}{std_v:>12.2f}")
print(f"{'Q1':<20}{g_q1:>12.2f}{q1:>12.2f}")
print(f"{'Q3':<20}{g_q3:>12.2f}{q3:>12.2f}")
print(f"{'IQR':<20}{g_iqr:>12.2f}{iqr_v:>12.2f}")
print("Modal class:", table['class'][m], "with frequency", f1)
```

```
Measure                   Grouped    Raw data
Mean                      2996.79     2987.53
Median                    2987.07     2968.50
Mode                      2921.05     2800.00
Variance                237732.10   237091.78
Std deviation              487.58      486.92
Q1                        2722.66     2735.50
Q3                        3319.85     3313.70
IQR                        597.20      578.20
Modal class: 2750-3000 with frequency 29
```

Observation: Grouped values are very close to the raw ones (mean 2996.79 vs 2987.53, std 487.58 vs 486.92). The small gap comes from using class midpoints instead of the real values. Modal class is 2750-3000 mm with 29 years.

Conclusion¶

Mean, median, mode, variance, standard deviation and IQR of Konkan & Goa rainfall were found using built-in functions, manual formulas and grouped data, and all three methods gave consistent results. Mean and median are nearly equal (about 2970-2990 mm) with a CV of 16%, so annual rainfall is stable. Monsoon (JJAS) rainfall drives the yearly total, while winter (JF) rainfall is highly variable.